School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00170371
Student Name:	Alexandru Belecci

Module Information

Module Title:	Security Administration
Module Code:	COMPH2703

Assessment Information

Assessment Title:	Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☑ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Belecci
Date:	02/12/2025

Table of Contents

- Introduction
- Architecture
- Deployments and Configurations
- Conclusion

Introduction

This report focuses on designing, deploying, securing a cloud-based infrastructure and applying Security Administration Principles that are operated on Microsoft Azure. The Ubuntu Virtual Server deployed for hosting the web application, Azure SQL Database for structured data storage, Azure Storage for storing files (photos in our case), a well-protected network with DDoS protection enabled for secure communication and rules.

The output of this project is the implementation of strong security controls that include network segmentation, access control rules, encryption, system hardening, monitoring logging and backup strategies. All this resulted in a practical approach to secure cloud resources by following best practices and principles of least privilege.

Architecture

The core of architecture is the Ubuntu Server, that runs the front end and the back end, handles the user authentication, and processes the application logic. To enforce traffic control, it was placed inside a subnet within the Virtual Network, and the server can only be accessed by the Host Machine.

Data is stored in SQL Database that resides in a separate subnet in the same VNET. To ensure that records are exchanged securely the direct public access was completely blocked and access to database is allowed only from VM.

Blob Storage used to store application files such as profile images used, which access is limited to a role assignment and storage level security.

The Network Security Groups control the inbound and outbound traffic, which allow specific ports to be accessed by administration TCP 22 (SSH) and for web access TCP 80/443, any other traffic is denied by default.

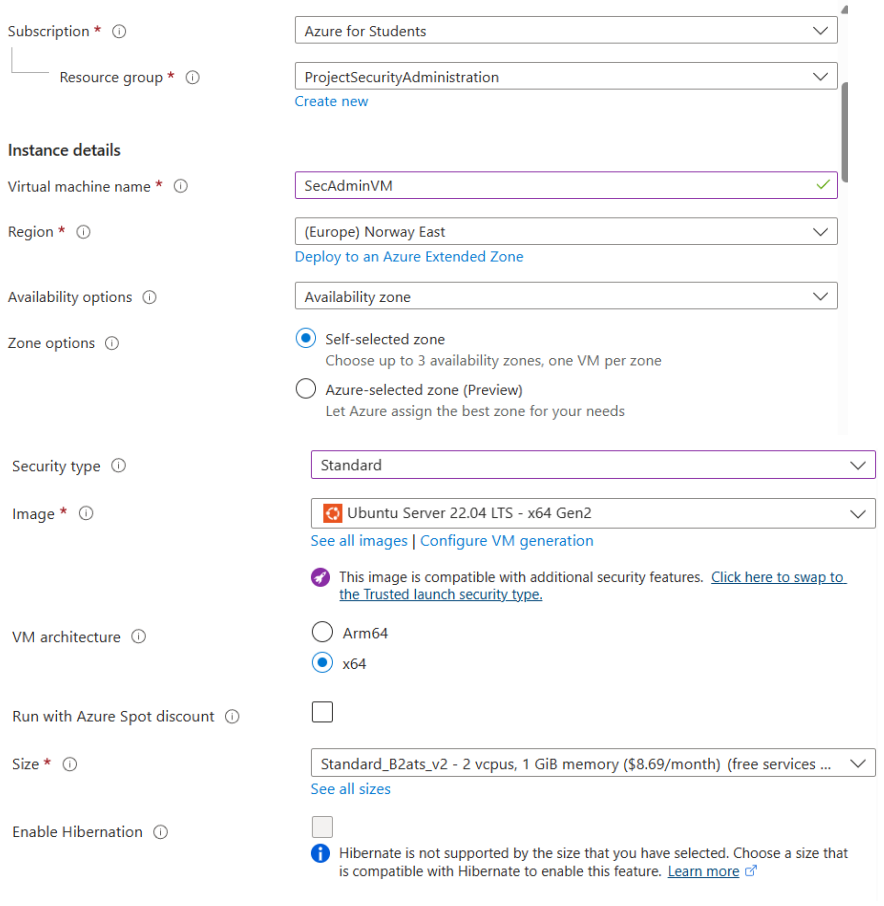
Ubuntu VM Server Creation-Configuration

First step of the project is creating the VM:

Using Azure for Students Subscription, in the resource group created for the project.

Region closest to us.

Everything else here default.



Security Type – Standard

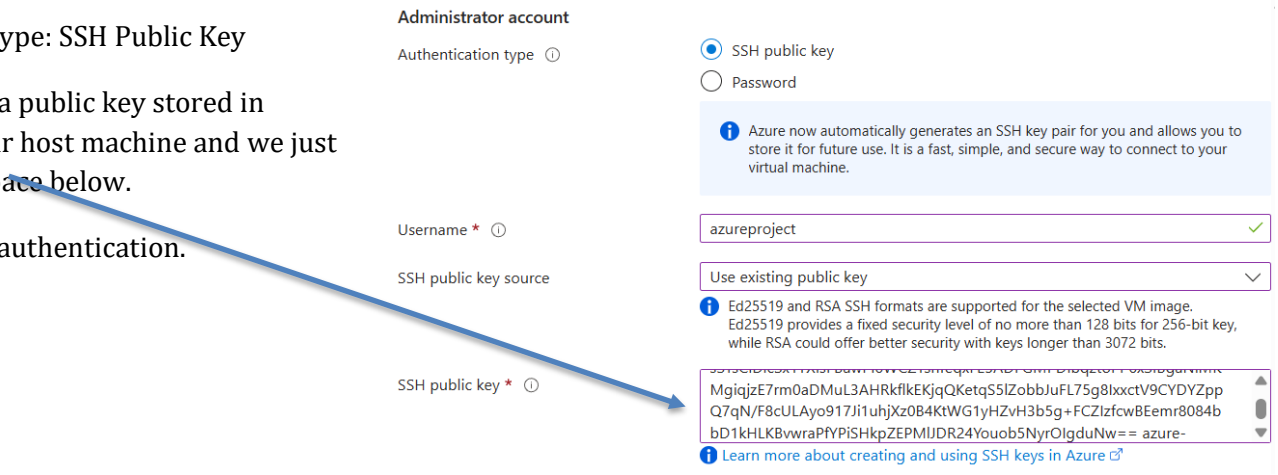
OS Image – Ubuntu Server 22.04 LTS (free 750h)

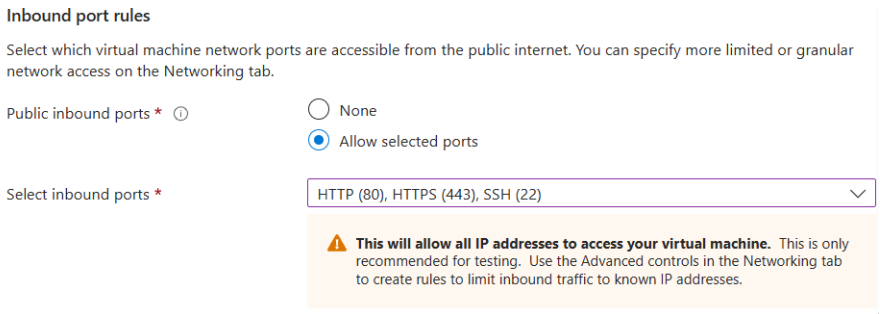
Size – Standard_B2ats_v2 (eligible for subscription).

Authentication type: SSH Public Key

And we created a public key stored in .ssh/id_rsa in our host machine and we just paste it in the space below.

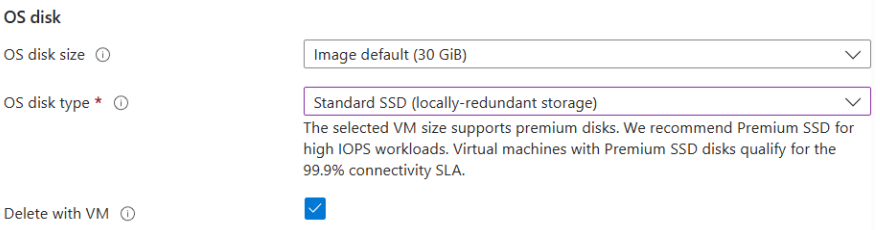
This will be our authentication.





VM Only Allows SSH, HTTP and HTTPS

Allow HTTP (80), HTTPS (443) and SSH (22) are going to be accessible from outside.



The Disk type required is Standard.

SSH Configuration

When SSH system asking for passphrase created to access the Authentication Key.

```
C:\Users\Admin>ssh project@20.251.153.190
Enter passphrase for key 'C:\Users\Admin/.ssh/id_rsa':
```

No Root/Admin SSH login allowed on Linux

When SSH successfully, we access file **/etc/ssh/sshd_config**

And check to make required changes.

To desiable password authentication and root access in system.

To restart configuration: **systemctl restart sshd.**

```
RSAAuthentication yes
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
```

PAM Configuration – Strong Password Configuration

Strengthening the system authentication, by reviewing the Linux PAM:

/etc/pam.d/common-auth – how system processes login attempts and ensures that only valid users can authenticate.

Pam_unix.so – secure password-based authentication

Pam_deny.so / pam_permit.so – correct authentication and prevents unauthorized users.

```
 GNU nano 6.2                 /etc/pam.d/common-auth
#
# /etc/pam.d/common-auth - authentication settings common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of the authentication modules that define
# the central authentication scheme for use on the system
# (e.g., /etc/shadow, LDAP, Kerberos, etc.).  The default is to use the
# traditional Unix authentication mechanisms.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules.  See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
auth    [success=1 default=ignore]      pam_unix.so nullok
# here's the fallback if no module succeeds
auth    requisite                       pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
auth    required                        pam_permit.so
# and here are more per-package modules (the "Additional" block)
auth    optional                        pam_cap.so
# end of pam-auth-update config
```

Strong password standards are enforced in **/etc/pam.d/common-password**

Pam_pwquality.so – is themodule that applies complexity rules eg. Minimum length, retries, new password differentiation.

```
# here are the per-package modules (the "Primary" block)
password        requisite pam_pwquality.so retry=3 minlen=12 difok=3 ucredit=1 lcredit=1 dcredit=1 ocredit=1
password        [success=1 default=ignore]      pam_unix.so obscure use_authtok try_first_pass yescrypt
```

Retry = 3 (3 attempts allowed),

Minlen = 12 (12+ characters password)

Difok = 3 (3 character differ from previous password)

Ucredit = 1 (1 uppercase)

Lcredit = 1 (1 lowercase)

Dcredit = 1 (1 digit)

Ocredit = 1 (1 symbol)

Yescrypt = password hashing

The file **/etc/pam.d/common-account** defines whether a user is allowed to authenticate.

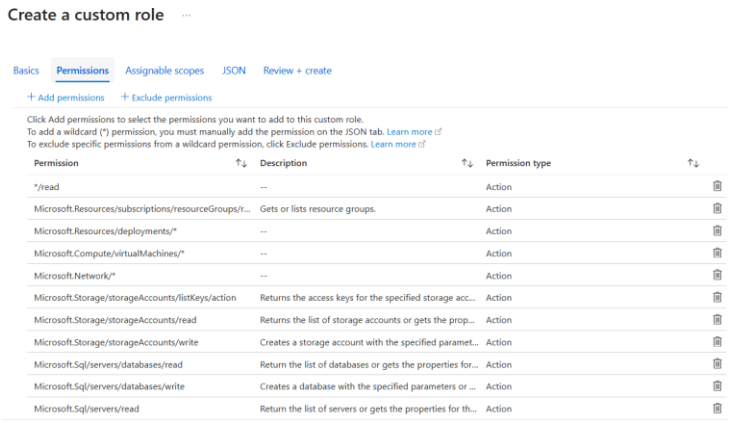
Allows only valid system accounts, check by pam_unix.so account status and pam_deny.so / pam_permit,so ensure control flowing.

```
# here are the per-package modules (the "Primary" block)
account [success=1 new_authtok_reqd=done default=ignore]        pam_unix.so
# here's the fallback if no module succeeds
account requisite                       pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
account required                        pam_permit.so
# and here are more per-package modules (the "Additional" block)
# end of pam-auth-update config
```

All this confirms that ssh is using PAM:

```
# Standard Un*x authentication.
@include common-auth
```

Azure RBAC

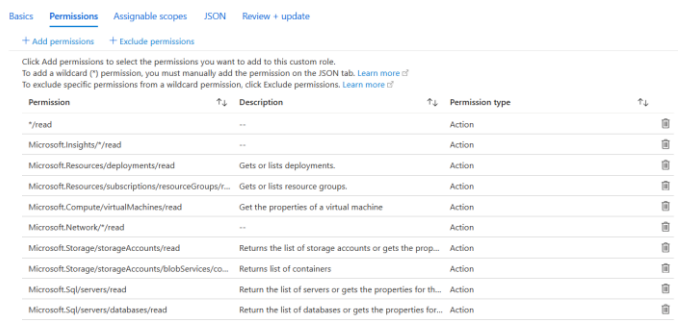


First Developers of the app will be granted azure RBAC custom role. Allowed to interact with the application environment without giving administrative privileges.

Their permissions are:

- Read resource configurations (*/read)
- Access virtual machines (app deployment) (Microsoft.Compute/virtualMachines/*)
- Interact with storage accounts
- Access database resources
- Network Resources (read and access) (Microsft.Network/*)

Update a custom role



For ProjectManagers, the RBAC role was assigned to give them enough permissions to be able to view and monitor the cloud environment.
Because of the need of broad visibility across services in order for tracking the progress, review deployments and verify that infrastructure is fully working.
The custom role includes only read permissions across everything from cloud.
This Least Privilege permissions allow them to keep an eye on everything.

- View VM Status and configurations
- Inspect NSG and topology
- View Database properties and connections
- Deployment History
- Access logs, metrics and alerts

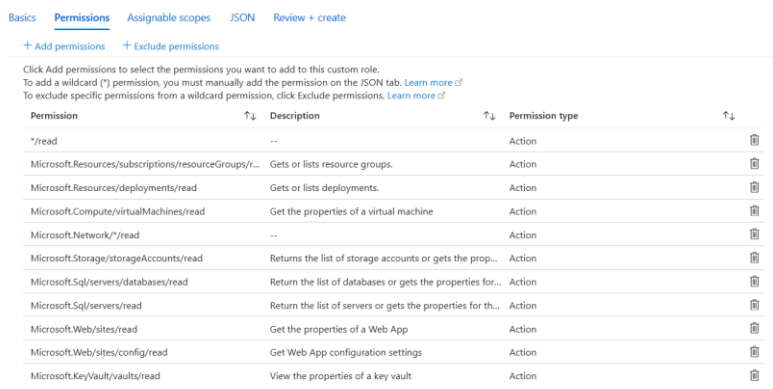
the custom RBAC role that grants QATesters read-only visibility across all resources required for application testing and validation.

They need to verify deployments, inspect application behavior, view logs and check resources statuses.

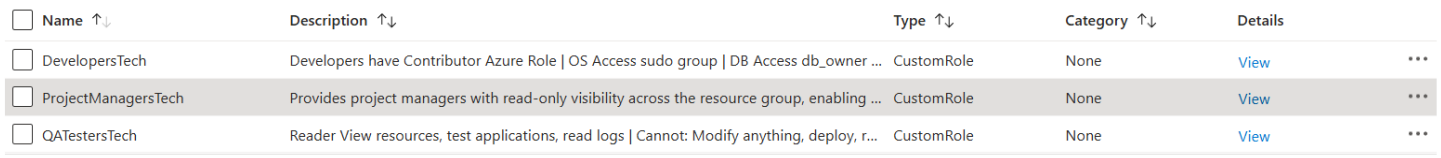
But should not be able to modify or delete.

- Read permissions for VMs, Storage accounts, SQL Database, Network Resources and deployment History.

Create a custom role



This allows them to observe the system's behavior and confirm that functioning correctly.



These roles were assigned in order to grant them only required permissions, be enforcing Least-Privilege model access to cloud environment. In order to minimize security risks and reduce chance of accidental misconfiguration(effective role-based access).

Appropriate Subnet for VM and Database

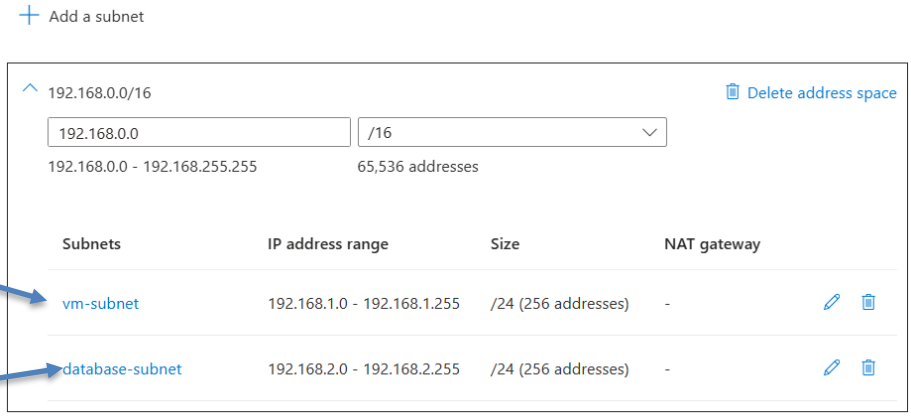
Proper Network segmentation within the Cloud environment, a virtual network was created (when create vm)

Inside I created a subnet for VM:

- Address range 192.168.1.0/24

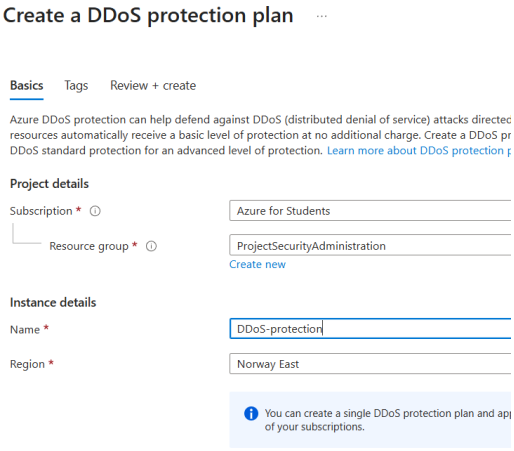
And for Database:

- Address range 192.168.2.0/24



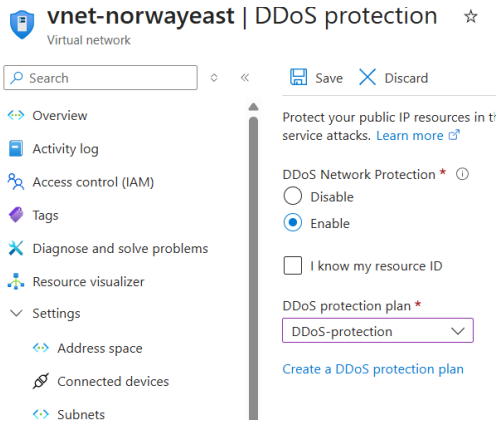
This ensures that data resources are isolated from each other and Traffic can be monitored and restricted using Network Security Group.

DDoS Protection



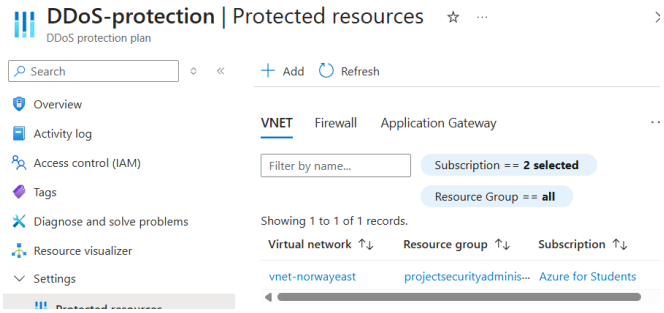
For enabling DDoS protection on our Network, firstly a DDoS Protection Plan should be created. Azure already provides basic DDoS protection by default, the custom configured plan enables an advanced level of protection:

- Adaptive Threat Detection
- Automated Mitigation
- Real-Time Traffic Monitor
- Etc.

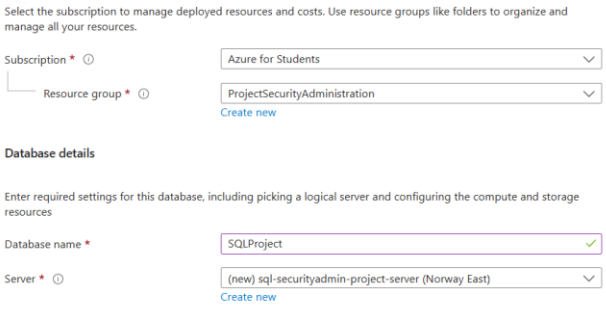


Then we go to our Virtual Network and in Settings look for DDoS protection. Enable it, select the Plan created earlier and save it.

And you can check if it worked by entering in the DDoS Protection Plan, and you will see at the bottom, your protected network.



SQL Database Creation-Configuration

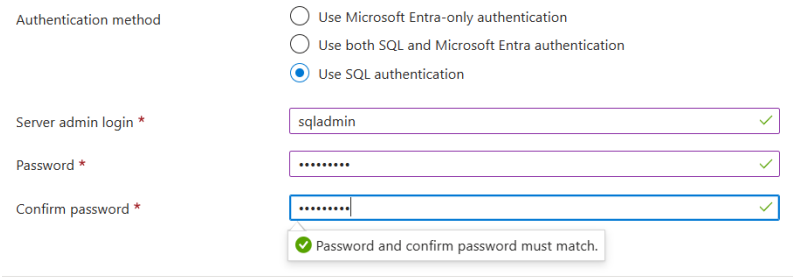


Using Azure for Students Subscription.

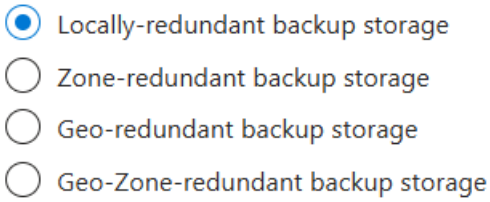
Placing the Database in the Resource Group for the project.

Creating a new server for it and in the server:

Authentication method select: Use SQL authentication

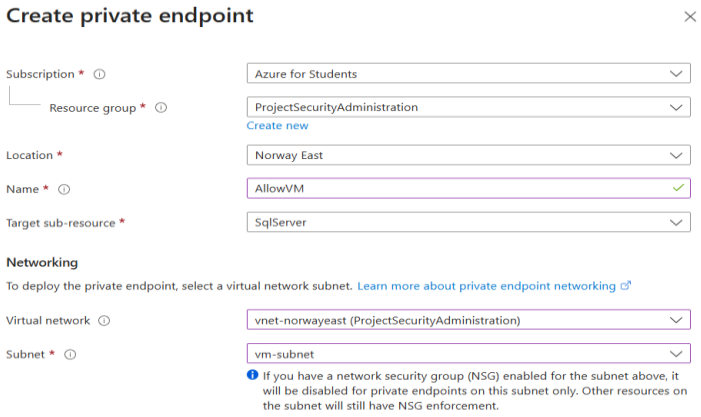


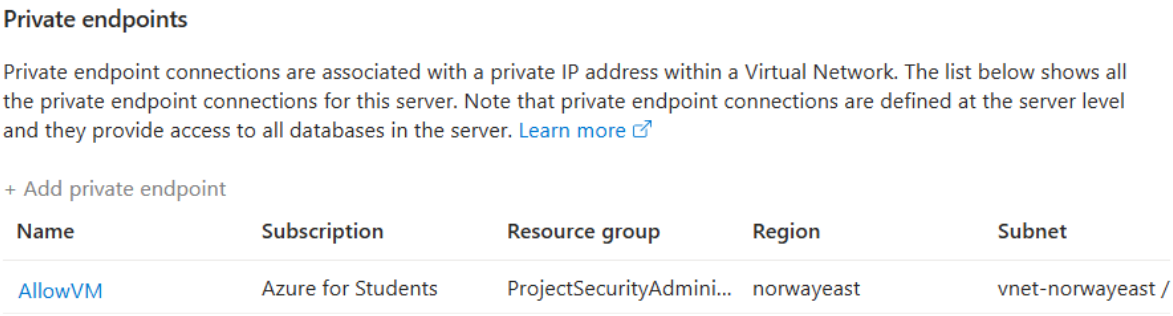
As method for backup: LRS



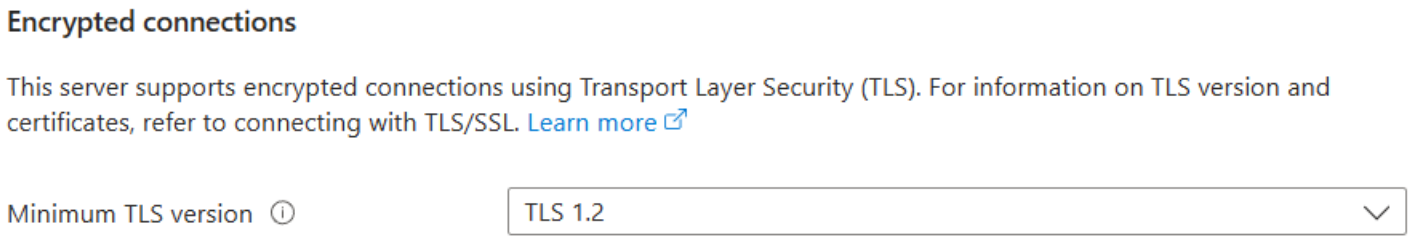
Database only accessible from VM

To ensure that the Azure Database is only accessible from VM and not accessible from Public, a Private Endpoint helps. Assigns a private IP address that allows only internal resources, in this case specifically VM.



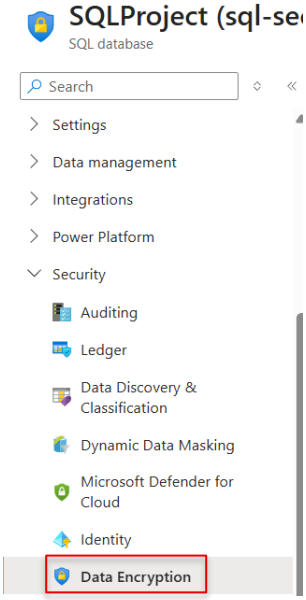


SQL Connection: SSL/TLS Enforcement Enabled



To secure communication between the application and the Azure Database, the minimum version of TLS 1.2 must be set. This provides encrypted communication between Database and Client and ensures that modern encryption protocols are used.

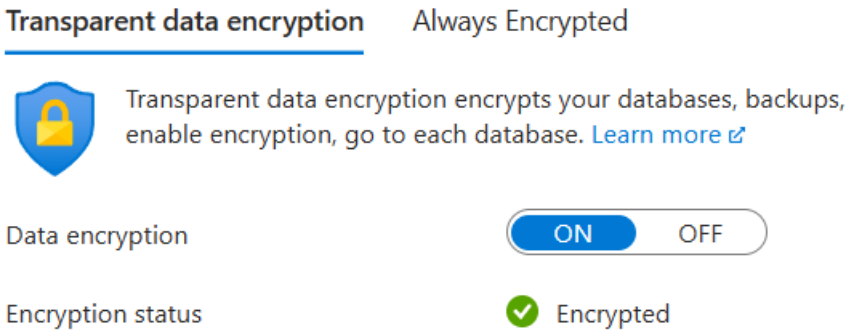
Azure SQL Database: TDE



In the left Menu of the Database Resource, under Security heading: Data Encryption

For protection:

- Data at Rest
- Database
- Backups
- Restores
- Etc.

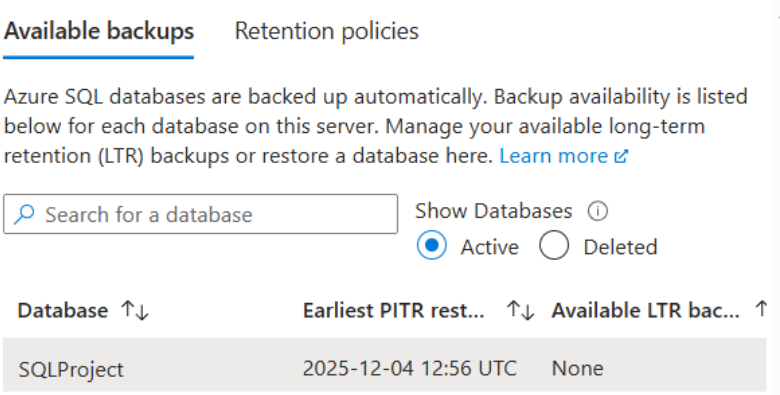


This ensures that if the physical storage or backup files were accessed, the database content remains unreadable.

Azure SQL Database Backups

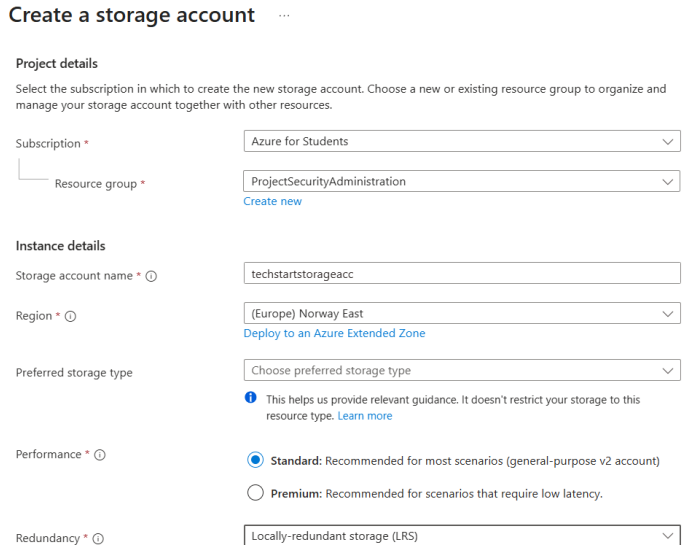
Includes built in automated backups, which ensures that a Database can be restored at any time in case of emergency:

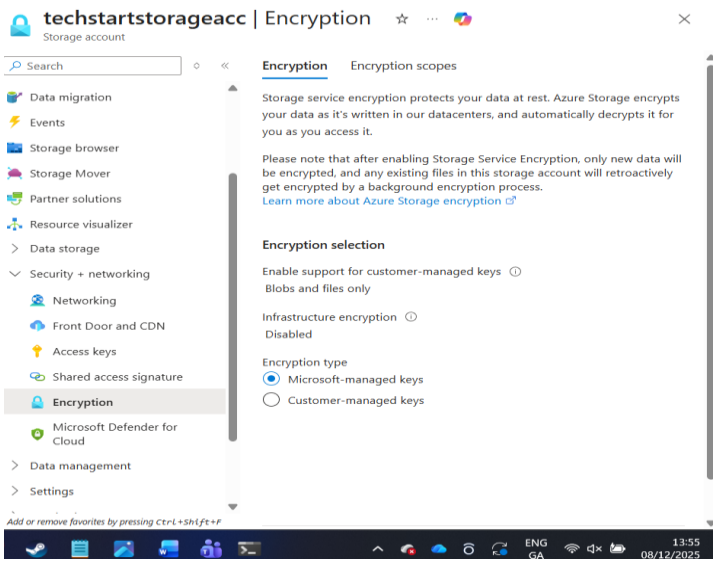
- Data loss, corruption, accidental modification



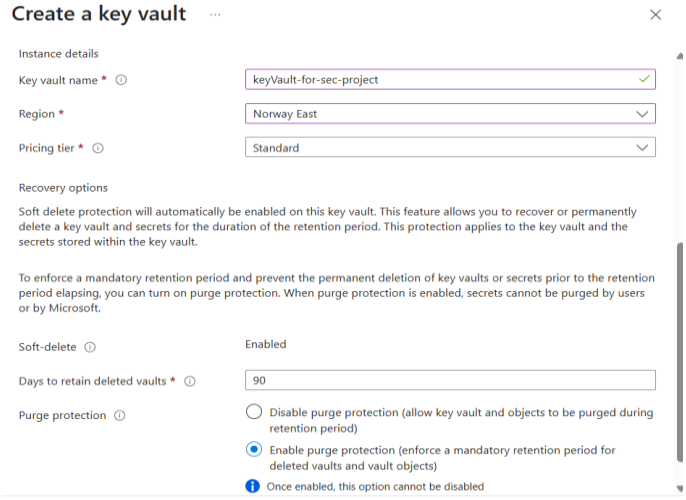
Azure Blob Storage: Encryption at rest enabled + Use Azure Key Vault

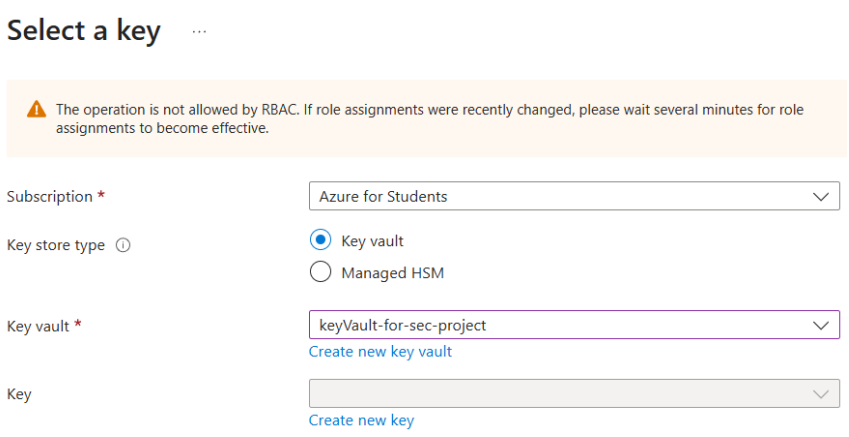
Azure Blob Storage automatically applies encryption at rest to all stored data (AES 256).





This can be checked after deploying it, in the left menu under Security + Networking heading >> Encryption
But we are always able to customize it by creating an Azure Key Vault Key and implement it to be used by storage Account.





To securely store Secrets for project, I created a custom Azure Key Vault named: keyVault-for-sec-project. Two features were enabled during deployment: Soft Delete which ensures that any delete secrets can be recovered for 90 days; And Purge Protection that blocks deletion of items until retention period expires.

These settings help by protecting critical secrets.

Configure NSG Rules (Principle of Least Privilege) + Block all other unnecessary ports on VM.

150	AllowSSH	22	TCP	64.43.50.13	Any	Allow	
200	ALLOWHttp	80	TCP	Any	Any	Allow	
210	ALLOWHttps	443	TCP	Any	Any	Allow	
300	AccessWEB	5000	TCP	64.43.50.13	Any	Allow	
4000	BlockEverything	Any	Any	Any	Any	Deny	

- AllowSSH only from my personal Public IP (port 22, protocol TCP, from 64.43.50.13, to any, Allow) 150 priority.
- ALLOWHttp and ALLOWHttps from any source enabling public access to application (port 80 and 443, protocol TCP, from any, to any Allow) 200/210 priority
- AccessWEB only from my Public IP for testing application (port 5000, protocol TCP, from 64.43.50.13, to any, Allow)300 priority
- BlockEverything Deny any inbound traffic (port any, protocol any, from any, to any, Deny) priority 4000.

UFW Configured

```
project@VMProject:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
80/tcp                     ALLOW       Anywhere                  # Allow HTTP
443/tcp                    ALLOW       Anywhere                  # Allow HTTPS
22/tcp                     ALLOW       64.43.50.97
5000/tcp                   ALLOW       64.43.50.97
80/tcp (v6)                ALLOW       Anywhere (v6)             # Allow HTTP
443/tcp (v6)               ALLOW       Anywhere (v6)             # Allow HTTPS
```

For securing the VM at the OS level, I configured the UFW using a default deny incoming policy. To ensure that the server blocks all unsolicited traffic unless an allow rule is added.

Port 22 TCP for SSH only open to my personal IP address, for administrative access.

Port 5000 TCP used for application testing also open only for my IP address.

Port 80 and 443 TCP open to public to make the application accessible.

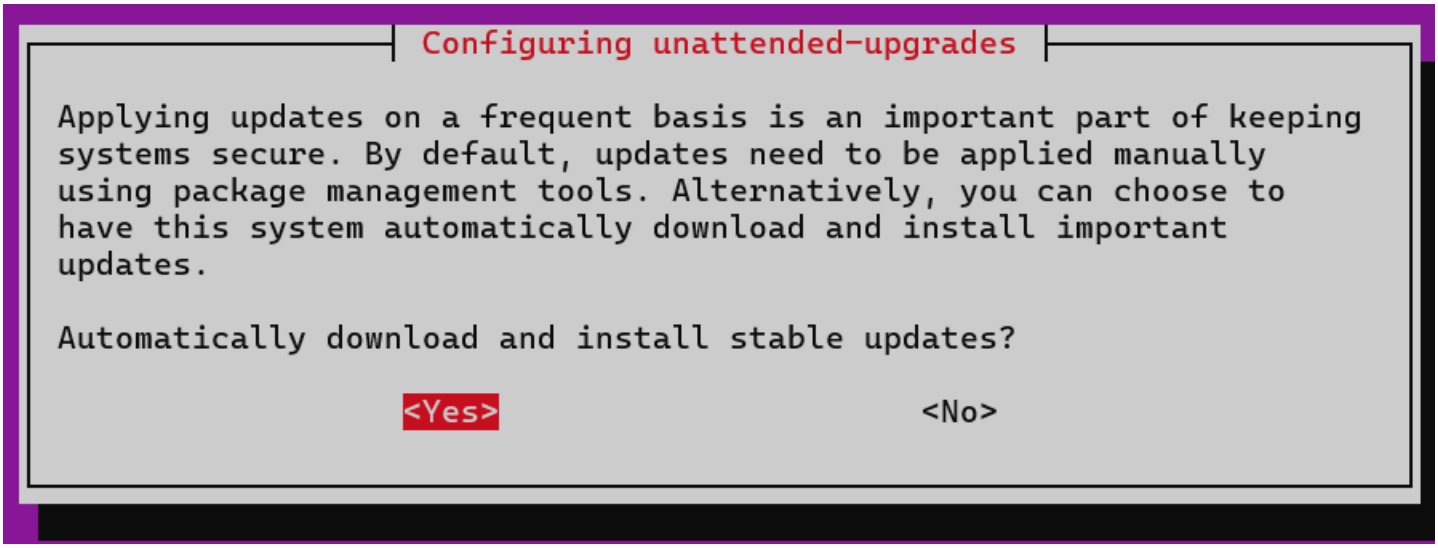
And enable the firewall using ‘sudo ufw enable’.

Automatic Security Updates Enabled

```
project@VMProject:~$ sudo apt install unattended-upgrades
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
unattended-upgrades is already the newest version (2.8ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
project@VMProject:~$ sudo dpkg-reconfigure -plow unattended-upgrades
```

To ensure that VM remains secure over time without manual updates, reducing the risk of vulnerabilities caused by outdated software, I configured unattended-upgrades. Service to automatically install security updates which allows the system to download and apply important patches.

This setup improves security posture of VM by ensuring that critical system components, libraries and services receive updates as soon as possible.



'Sudo dpkg-reconfigure -plow unattended-upgrades' – enables the automatic updates by pressing 'yes' at the window prompted.

Block SSH brute force attempts

To provide protection against brute force SSH attacks, Fail2ban tool was installed. It monitors authentication logs for repeated failed login attempts and blocks automatically access of that specific IP for a period of time.

This helps against password-guessing tools and reduces the risk of unauthorized access.

'sudo apt install fail2ban' – to install tool.

```
^C(venv) project@VMProject:~/taskapp$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban
ail.local
[sudo] password for project:
Sorry, try again.
[sudo] password for project:
(venv) project@VMProject:~/taskapp$ sudo nano /etc/fail2ban/jail.local
```

I copied the default configuration file 'jail.conf' (/etc/fail2ban/jail.conf) into /etc/fail2ban/jail.local for future configuration.

```
[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 3600
findtime = 600
# See jail.conf(5) man page for more information
```

- SSHD was enabled.
- After 3 failed login attempts ban the IP for 1 hour.
- Track failed attempts 10 min
- Monitor SSH authentication logs

```
(venv) project@VMProject:~/taskapp$ sudo systemctl restart fail2ban
sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset:>
     Active: active (running) since Sun 2025-12-07 21:12:51 UTC; 22ms ago
```

Restart Fail2ban to update configurations and to access the 'tool logs' – sudo fail2ban-client status sshd

```
(venv) project@VMProject:~/taskapp$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
```

Here you can always check if any suspicious activity regarding SSH happened.

System Logging Configured + Log Retention Configured

```
project@VMProject:~$ sudo systemctl status rsyslog
● rsyslog.service - System Logging Service
     Loaded: loaded (/lib/systemd/system/rsyslog.service; enabled; vendor p>
     Active: active (running) since Sun 2025-11-30 10:17:28 UTC; 1h 4min ago
TriggeredBy: ● syslog.socket
       Docs: man:rsyslogd(8)
             man:rsyslog.conf(5)
             https://www.rsyslog.com/doc/
   Main PID: 562 (rsyslogd)
      Tasks: 4 (limit: 1069)
     Memory: 2.9M
        CPU: 98ms
     CGroup: /system.slice/rsyslog.service
             └─562 /usr/sbin/rsyslogd -n -iNONE

Nov 30 10:17:28 VMProject systemd[1]: Starting System Logging Service...
Nov 30 10:17:28 VMProject rsyslogd[562]: imuxsock: Acquired UNIX socket '/r>
Nov 30 10:17:28 VMProject rsyslogd[562]: rsyslogd's groupid changed to 111
                                         changed to 104
                                         rsyslogd" swVers>
                                         g Service.
```

System logging on the Linux is implemented using rsyslog.

Responsible for collecting system logs and keeping them into /var/log/ directory.

To verify if system logging was correctly configured and functionable youcan check status:

'Sudo systemctl status rsyslog'

Showing that service is active and starts automatically at boot, confirming that logging is continuously operational.

```
/var/log/syslog
/var/log/mail.info
/var/log/mail.warn
/var/log/mail.err
/var/log/mail.log
/var/log/daemon.log
/var/log/kern.log
/var/log/auth.log
/var/log/user.log
/var/log/lpr.log
/var/log/cron.log
/var/log/debug
/var/log/messages
{
        rotate 4
        weekly
        missingok
        notifempty
        compress
        delaycompress
        sharedscripts
        postrotate
                /usr/lib/rsyslog/rsyslog-rotate
        endscript
}
```

Additionally, to log collection, a retention policy is configured using logrotate. It manages the duration of logs kept in system and prevents uncontrolled log growth.

- Rotates, compresses and removes old log files following defined rules.

'sudo vim /etc/logrotate.conf' to check and make changes.

- Logs are rotated ona a weekly basis
- 4 copies retained
- Old logs compressed to reduce disk usage.

In addition to default system log retention, I created a custom logrotate policy for application specific logs. Rotates the web_backup.log on a daily basis, keeps 5 copies and ensures new log files are created with secure permissions. It keeps control over log retention.

```
/var/log/web_backup.log {
    daily
    rotate 5
    compress
    missingok
    notifempty
    create 0640 root root
}
```

OS Level Backup

To ensure data availability and recoverability, OS level backup policy was implemented on Linux VM.

Custom script created to backup the web application directory, which generates timestamped backup directories in /var/backups. It allows recovery if any of these occur:

- Deletion, corruption, system failure.

```
#!/bin/bash

BACKUP_DIR="/var/backups/webapp_$(date +%F_%H-%M-%S)"
mkdir -p "$BACKUP_DIR"


cp -r /home/project/taskapp "$BACKUP_DIR"


echo "Backup completed at $(date)" >> /var/log/web_backup.log
```

This provides visibility to backup activity and allows view if completed successfully.

'sudo crontab -e'

```
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

0 6 * * * tar -czf /var/backups/taskapp_$(date +\%F).tar.gz /home/project/taskapp
```

This scheduled task runs daily at 06:00 automatically,only by configuring a cron job.

Backup Verification Script

Bash script chosen was Backup verification script, it verifies if backups run successfully, which is critical security control.

It automatically checks if a recent backup file exists in the system and verifies that the backup was created in a 24-hour time window. This ensures that backups are produced and remain usable.

- Confirms backup file is present in /var/backups
- File size > 0
- Checks if file is recent
- Logs success or failure result

This script removes the need of manual verification and ensures fast detection of any failures.

Each execution of script appends a timestamped result to log file.

‘sudo nano backup_check.sh’

```
#!/bin/bash

LOG_FILE="/var/log/backup_check.log"
BACKUP_PATH="/var/backups/taskapp_$(date +%F).tar.gz"


tar -czf "$BACKUP_PATH" /home/project/taskapp

echo "===== Backup Check $(date) =====" >> "$LOG_FILE"


if [ ! -f "$BACKUP_PATH" ]; then
    echo "ERROR: Backup file not found: $BACKUP_PATH" >> "$LOG_FILE"
    exit 1
fi


FILE_SIZE=$(stat -c%s "$BACKUP_PATH")
if [ "$FILE_SIZE" -le 0 ]; then
    echo "ERROR: Backup file is empty." >> "$LOG_FILE"
    exit 1
fi


MAX_AGE=$((24 * 60 * 60))
CURRENT_TIME=$(date +%s)
FILE_MOD_TIME=$(stat -c%Y "$BACKUP_PATH")
FILE_AGE=$((CURRENT_TIME - FILE_MOD_TIME))

if [ "$FILE_AGE" -gt "$MAX_AGE" ]; then
    echo "ERROR: Backup is older than 24 hours." >> "$LOG_FILE"
else
    echo "SUCCESS: Backup is recent and valid." >> "$LOG_FILE"
fi

echo "Backup check finished." >> "$LOG_FILE"
```

‘sudo ./backup_script.sh’

```
project@VMProject:~/taskapp$ sudo ./backup_script.sh
tar: Removing leading `/' from member names
```

‘sudo cat /var/log/backup_check.log’

```
===== Backup Check Fri Dec 12 17:39:07 UTC 2025 =====
SUCCESS: Backup is recent and valid.
Backup check finished.
```

The log contains message SUCCESS: Backup is recent and valid. Means that it run successfully and file is good for operation.

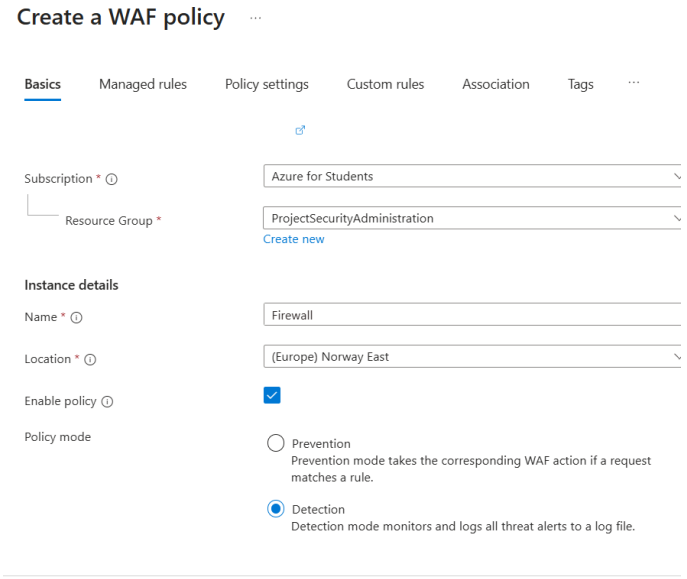
WAF

A Web Application Firewall policy was created to protect applications against common web attacks. This provides application layer 7 security by inspecting incoming HTTP and HTTPS requests before they reach the Server.

Set to Detection mode.

It supports a defense-in-depth approach:

- Detecting attacks
- Monitoring Bot activity
- Visibility into Malicious traffic



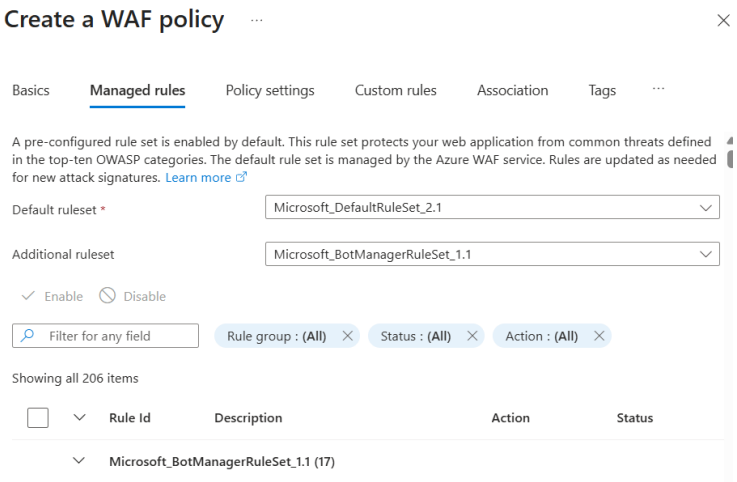
Managed Rule

- Microsoft_DefaultRuleSet_2.1

  Protects application against common vulnerabilities in the OWASP.

- Microsoft_BotManagerRuleSet_1.1

  Protection against malicious and automated Bot traffic.



TLS Encryption For Application

HTTPS is implemented on Linux using Nginx and TLS for securing communication between users and applications. On port 443 the connection is managed by Nginx and internal communication remains at the local host to reduce exposure.

- Nginx redirects all http traffic to port 80.
- No unencrypted connection
- Protected sensitive data

```
project@VMProject:~$ sudo ln -s /etc/nginx/sites-available/flaskapp /etc/nginx/site
s-enabled/
project@VMProject:~$ sudo systemctl restart nginx
```

This ensures confidentiality and integrity of data.

'sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048'

Generated a self-signed certificate using OpenSSL with a 2048bit RSA key, in order encrypted connection to be enabled.

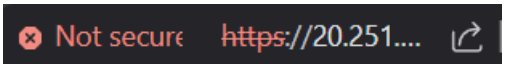
```
server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/ssl/flask/selfsigned.crt;
    ssl_certificate_key /etc/ssl/flask/selfsigned.key;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

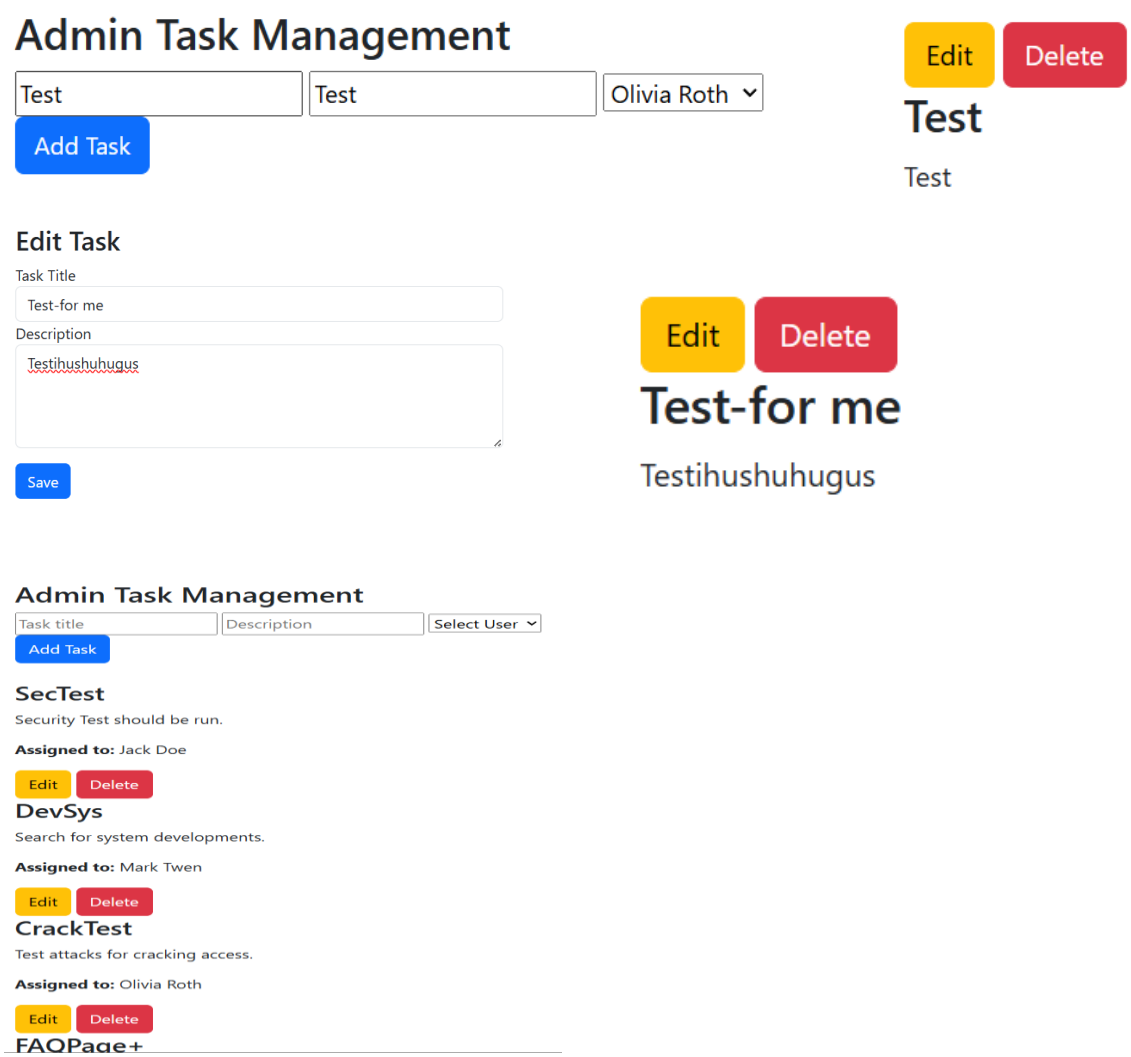
server {
    listen 80;
    return 301 https://$host$request_uri;
}
```

Open Nginx configuration to implement changes. 'sudo nano /etc/nginx/sites-available/flaskapp'

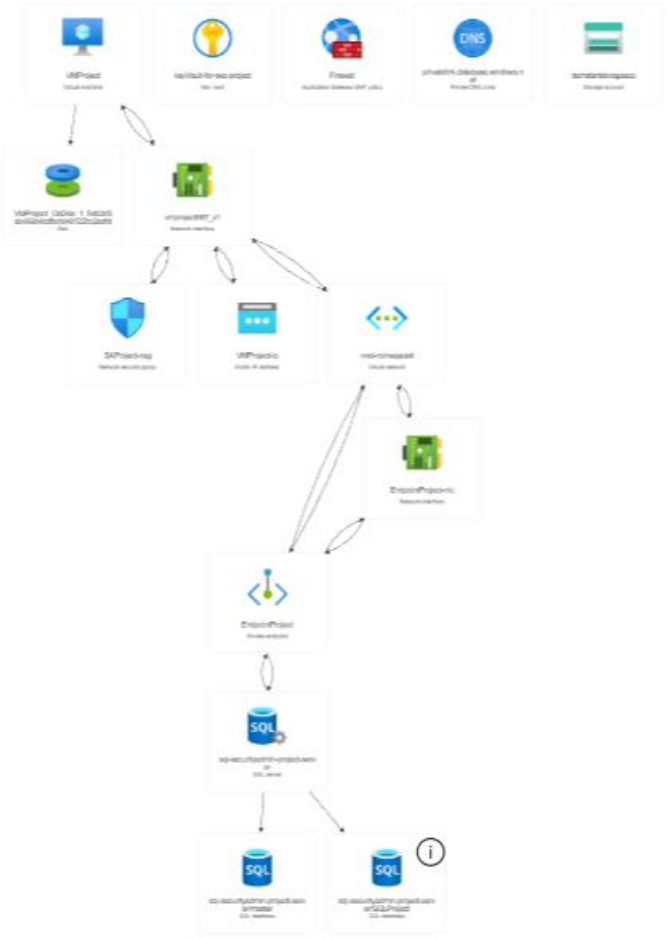


And web application running on HTTPS.

APP performing Add, Delete, Edit



Resource Visualization



This visualization provides a detailed resource view, generated by Azure showing all deployed cloud resources and connections between them.

- VM hosts the web application.
- Associated Network interfaces and Public IP
- NSG with the filtering rules implemented
- Azure VNet and subnets
- Private endpoint for accessing Azure SQL

Diagram demonstrates that database connectivity is established through a private endpoint, this ensures that database traffic is not exposed to internet.

The connection paths prove that VM communicates through a Private IP with the database, while everything else is restricted by the Network Security Group.



Conclusion

This project involved the deployment of a secure cloud environment in which an application is hosted on a Ubuntu Virtual Machine within Microsoft Azure. A virtual network is a core design of the infrastructure with separate subnets for application and database components, for network isolation and principle of least privilege. Network Security Groups were configured to block the unnecessary traffic coming towards and out of the network.

Data security is ensured through encryption at rest and movement, and additional security measures such as system hardening, logging, backups and automated security task are implemented for providing a secure cloud deployment.